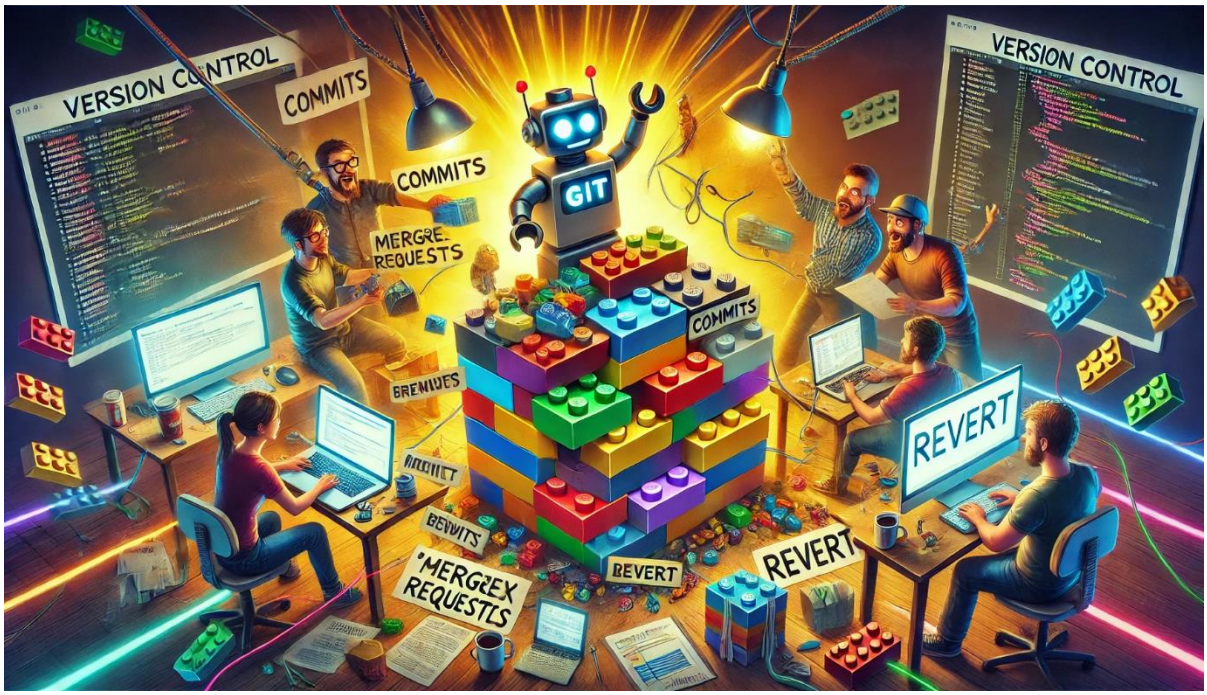
INTRODUCCIÓN AL CONTROL DE VERSIONES



Javier López Gutiérrez

Índice:

- ¿Qué es el control de versiones?
- Clasificación de los sistemas de control de versiones
- Características de un sistema de control de versiones
- Ventajas
- Sistemas de control de versiones. Incluye una tabla con los más conocidos y una descripción.
- ¿Qué son los conflictos?
- Introducción a GitHub: ventajas, características, funciones de revisión.

¿Qué es el control de versiones?

El control de versiones es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, lo que permite que puedas rastrear y revertir modificaciones, colaborar con otros y administrar el progreso de un proyecto de manera eficiente.

Características clave:

Seguimiento de cambios: Cada modificación a los archivos se registra, junto con información sobre quién la hizo y cuándo.

Colaboración: Permite que varios desarrolladores trabajen en un proyecto al mismo tiempo sin sobrescribir el trabajo de otros.

Reversión: Puedes regresar a versiones anteriores de un archivo o proyecto si algo sale mal.

Ramas y fusiones: Puedes trabajar en diferentes funciones o correcciones de errores en paralelo usando ramas, y luego combinarlas en la versión principal del proyecto.

Tipos de sistemas de control de versiones: Local: Todo el historial de cambios se guarda en tu máquina local. Ejemplo: copias manuales en diferentes carpetas.

Centralizado: Hay un único servidor que almacena todas las versiones de los archivos, y los usuarios descargan y suben cambios.

Ejemplo: Subversión (SVN). Distribuido: Cada usuario tiene una copia completa del historial del proyecto en su máquina, lo que permite trabajar incluso sin conexión.

Ejemplo: Git.

Ejemplo práctico: Con Git, puedes Crear un repositorio para almacenar tu proyecto (git init). Realizar un seguimiento de los cambios (git add, git commit).

Colaborar con otros mediante plataformas com GitHub o GitLab

o

Clasificación de los sistemas de control de versiones:

Sistemas de Control de Versiones Locales
Descripción: Los cambios se rastrean y almacenan en el sistema de archivos de la máquina local del usuario.
Funcionamiento: Se crea un historial de versiones en el disco duro del usuario, usualmente mediante copias manuales o herramientas específicas como RCS (Revision Control System).
Ventajas: Fácil de implementar. Bajo consumo de recursos.
Desventajas: No permite colaboración entre múltiples usuarios.
Alto riesgo de pérdida de datos si se daña la máquina local.
Ejemplo: RCS.2. Sistemas de Control de Versiones Centralizados (SCVC)Descripción: Existe un servidor central que almacena todos los archivos y su historial de cambios. Los usuarios se conectan al servidor para obtener la última versión o enviar cambios.
Funcionamiento: Los desarrolladores descargan la última versión del repositorio al iniciar su trabajo. Después de realizar modificaciones, suben sus cambios al servidor.
Ventajas: Facilita la colaboración entre usuarios.
Control centralizado de versiones.
Ideal para equipos pequeños o medianos.
Desventajas: Dependencia de la conexión al servidor (si el servidor falla, nadie puede acceder al historial ni colaborar).Más lento en comparación con sistemas distribuidos.
Ejemplos: Subversión (SVN), Perfore, CVS.3. Sistemas de Control de Versiones Distribuidos (SCVD)Descripción: Cada usuario tiene una copia completa del historial del proyecto en su máquina local, lo que permite trabajar sin necesidad de una conexión permanente al servidor.
Funcionamiento: Los desarrolladores clonan un repositorio completo (incluyendo todo el historial de cambios).Los cambios locales se realizan y se registran en la máquina del usuario. Para colaborar, los cambios se sincronizan con un repositorio remoto cuando sea necesario.

Ventajas: Permite trabajar sin conexión. Redundancia del historial: si el servidor remoto falla, cualquier copia local puede restaurarlo. Ideal para proyectos grandes o distribuidos.
Desventajas: Puede ser más complejo para principiantes. Requiere más espacio en disco debido a la copia completa del historial. Ejemplos: Git, Mercurial, Bazaar.

Características de un sistema de control de versiones

Un sistema de control de versiones (SCV) tiene varias características esenciales que lo hacen útil para gestionar proyectos, especialmente en desarrollo de software. Estas características son comunes tanto en sistemas centralizados como distribuidos.

1. Seguimiento de cambios Permite registrar los cambios realizados en los archivos a lo largo del tiempo. Cada cambio incluye detalles como: Qué se modificó. Quién realizó la modificación. Cuándo se hizo. Un mensaje de descripción (comentario).

2. Administración de versiones Almacena diferentes versiones o "snapshots" de los archivos. Puedes: Acceder a versiones anteriores. Comparar versiones. Restaurar versiones específicas en caso de errores.

3. Trabajo colaborativo Facilita que múltiples usuarios trabajen en un mismo proyecto. Maneja conflictos cuando varios desarrolladores editan el mismo archivo al mismo tiempo. Permite realizar cambios en paralelo mediante ramas.

4. Ramas y fusiones (Branches & Merges) Ramas: Puedes crear líneas de desarrollo separadas para trabajar en diferentes características o correcciones. Fusiones: Combina cambios de diferentes ramas en una sola, integrando el trabajo de todos los colaboradores.

5. Acceso remoto y sincronización En sistemas centralizados y distribuidos, puedes: Acceder al repositorio desde cualquier lugar. Sincronizar tus cambios con un servidor remoto o con otros usuarios.

6. Registro de auditoría Mantiene un historial detallado de todas las acciones realizadas. Útil para :Rastrear errores. Identificar la fuente de problemas. Revisar el progreso del proyecto.

7. Manejo de conflictos Detecta y resuelve conflictos cuando múltiples usuarios editan la misma parte de un archivo. Proporciona herramientas para: Fusionar cambios manual o automáticamente. Mantener la integridad del proyecto.

8. Compatibilidad con múltiples formatos Funciona con cualquier tipo de archivo, como: Código fuente. Imágenes. Documentos.

9. Seguridad Los cambios están protegidos contra modificaciones no autorizadas. Los repositorios pueden configurarse con: Control de acceso basado en roles. Autenticación y encriptación.

10. Escalabilidad Adaptable para proyectos pequeños o grandes, y para equipos pequeños o distribuidos globalmente. Ejemplo práctico de características en Git: Crear un repositorio: git init. Realizar un commit: git commit -m "Mensaje del cambio". Crear una rama: git branch nueva-rama.Fusionar ramas: git merge nueva-rama. Sincronizar con un repositorio remoto: git push origin main.

Ventajas:

Los sistemas de control de versiones (**SCV**) ofrecen numerosas ventajas que los convierten en herramientas fundamentales para el desarrollo de software y la gestión de proyectos. Aquí te detallo las principales:

1. Seguimiento de cambios

    Ventaja: Puedes registrar y rastrear todas las modificaciones realizadas en un archivo o proyecto.

    Beneficio: Permite entender quién hizo qué cambios, cuándo y por qué, ayudando a identificar errores o restaurar versiones anteriores.

2. Colaboración eficiente

    Ventaja: Varios desarrolladores pueden trabajar en el mismo proyecto simultáneamente.

    Beneficio: Facilita la integración del trabajo en equipo y permite manejar proyectos distribuidos.

3. Trabajo en paralelo (uso de ramas)

    Ventaja: Los desarrolladores pueden trabajar en diferentes funcionalidades o correcciones en ramas independientes.

    Beneficio: Evita interferencias entre los cambios y facilita la integración de diferentes líneas de trabajo.

4. Restauración de versiones

Ventaja: Puedes volver a cualquier versión previa de un archivo o proyecto.

Beneficio: Es útil para corregir errores, deshacer cambios problemáticos o comparar versiones.

5. Resolución de conflictos

Ventaja: Detecta y resuelve conflictos cuando varios usuarios modifican los mismos archivos.

Beneficio: Garantiza que los cambios sean consistentes y no sobrescriban el trabajo de otros.

6. Seguridad

- **Ventaja**: Los cambios están protegidos con un registro histórico y, en sistemas distribuidos, cada usuario tiene una copia completa del proyecto.
- **Beneficio**: Reduce el riesgo de pérdida de datos y permite la recuperación en caso de fallas en el servidor.

7. Trabajo sin conexión (en SCV distribuidos)

- **Ventaja**: En sistemas como Git, puedes trabajar sin estar conectado a un servidor.
- **Beneficio**: Incrementa la productividad en entornos donde no siempre hay conexión.

8. Compatibilidad con múltiples proyectos

- **Ventaja**: Un sistema de control de versiones puede manejar varios proyectos simultáneamente.
- **Beneficio**: Centraliza la gestión de versiones y facilita la organización.

9. Integración con herramientas modernas

- **Ventaja**: Los SCV como Git se integran con plataformas como GitHub, GitLab o Bitbucket.
- **Beneficio**: Facilita la colaboración global, el seguimiento de problemas (issues), y la automatización de pruebas (CI/CD).

10. Escalabilidad

- **Ventaja**: Los SCV pueden manejar proyectos pequeños y grandes con equipos pequeños o distribuidos globalmente.
- **Beneficio**: Es ideal para proyectos que crecen con el tiempo, como software empresarial o de código abierto.

11. Ahorro de tiempo y costos

- **Ventaja**: Automatiza procesos como la fusión de cambios y la resolución de conflictos.
- **Beneficio**: Reduce el tiempo necesario para administrar el proyecto y minimiza errores costosos.

12. Registro detallado del proyecto

- **Ventaja**: Mantiene un historial completo de todas las modificaciones realizadas.
- **Beneficio**: Proporciona transparencia y documentación implícita del desarrollo.

Ejemplo de uso práctico

Si un equipo de desarrollo trabaja en una nueva función:

1. **Crea una rama** para trabajar en la función (`git branch nueva-funcion`).
2. Cada desarrollador **hace commits** para registrar sus avances (`git commit`).
3. Cuando se completa, la función se **fusiona** con la rama principal (`git merge`).
4. Sistemas de control de versiones. Incluye una tabla con los más conocidos y una

Sistemas de Control de Versiones

Un sistema de control de versiones (SCV) es una herramienta que permite registrar, gestionar y coordinar los cambios realizados en archivos de un proyecto. Estos sistemas pueden ser locales, centralizados o distribuidos, y se utilizan ampliamente en el desarrollo de software y otras áreas donde se necesita un control detallado de versiones.

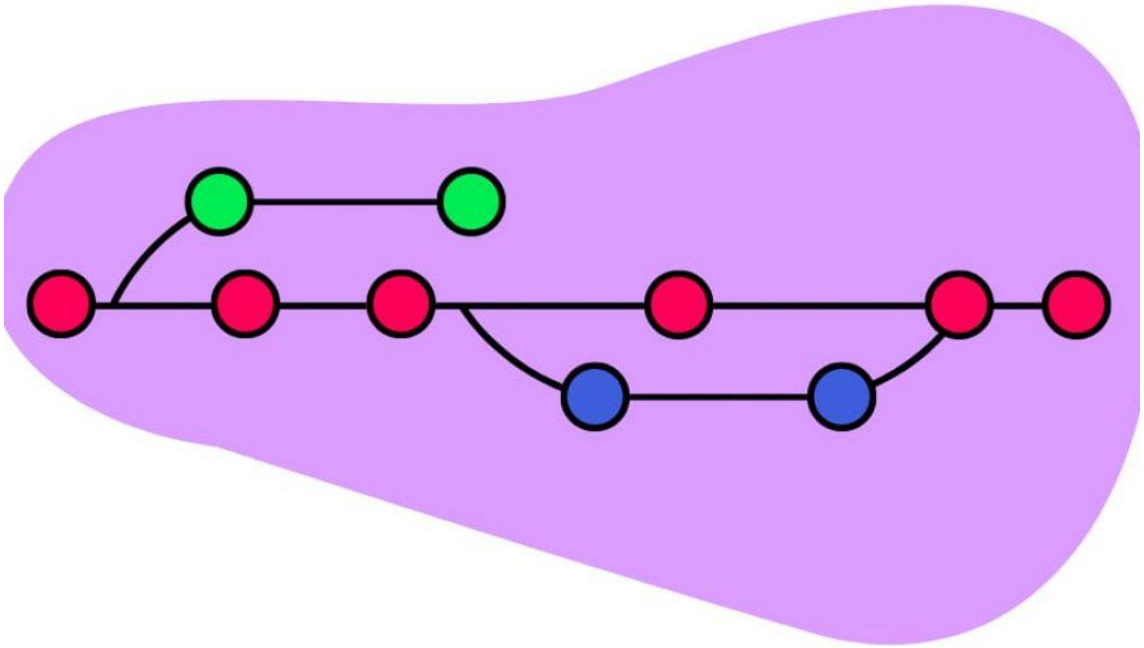


Tabla de los Sistemas de Control de Versiones Más Conocidos

Sistema	Tipo	Descripción	Ventajas Principales
Git	Distribuido	Sistema de código abierto diseñado para ser rápido y eficiente. Utiliza un modelo distribuido donde cada desarrollador tiene una copia completa del repositorio.	Escalabilidad, trabajo sin conexión, y amplia adopción gracias a plataformas como GitHub y GitLab.
Subversion (SVN)	Centralizado	Sistema centralizado que permite mantener versiones de archivos y directorios. Popular en proyectos legacy.	Fácil de aprender, historial detallado por archivo, y soporte para proyectos grandes.
Mercurial	Distribuido	Similar a Git, pero con enfoque en simplicidad y usabilidad. Utilizado en proyectos como Python antes de cambiar a Git.	Simplicidad, facilidad de uso y buen rendimiento.
Perforce (Helix)	Centralizado y Distribuido	Ofrece opciones tanto centralizadas como	Rendimiento alto, adecuado para archivos

Sistema	Tipo	Descripción	Ventajas Principales
		distribuidas, enfocado en proyectos grandes como videojuegos.	grandes, y excelente manejo de colaboración en equipo.
ClearCase	Centralizado	Sistema utilizado en grandes empresas, especialmente en proyectos con requisitos complejos de configuración y auditoría.	Soporte para configuraciones complejas y un sólido sistema de control de acceso.
Bazaar	Distribuido	Sistema flexible que permite tanto flujos de trabajo centralizados como distribuidos.	Flexible, fácil de usar, y soporte para múltiples modelos de desarrollo.
CVS (Concurrent Versions System)	Centralizado	Uno de los primeros sistemas de control de versiones ampliamente adoptados. Actualmente se considera obsoleto.	Simple y adecuado para pequeños proyectos.

Descripción de los Más Populares

Git:

Creado por Linus Torvalds para el desarrollo del kernel de Linux.

Características como las ramas ligeras y el trabajo sin conexión lo han convertido en el estándar de la industria.

Utilizado en plataformas como **GitHub**, **GitLab**, y **Bitbucket**.

Subversión (SVN):

Ideal para proyectos centralizados donde se necesita un control estricto sobre los archivos.

Su estructura jerárquica facilita la administración de archivos y directorios.

Mercurial:

Enfoque en ser más fácil de usar que Git, pero menos popular en la actualidad.

Sigue siendo utilizado en algunos proyectos legacy.

Perforce (Helix):

Popular en industrias que trabajan con archivos grandes, como videojuegos o diseño CAD.

Tiene herramientas avanzadas para la gestión de proyectos complejos.

Bazaar:

Desarrollado por Canonical (la empresa detrás de Ubuntu).

Su flexibilidad permite adaptarse a varios modelos de desarrollo.

CVS:

Aunque obsoleto, fue el estándar de facto en sus inicios.

Su simplicidad lo hace adecuado para proyectos pequeños y educativos.

¿Qué son los conflictos?

En un sistema de control de versiones, **los conflictos** ocurren cuando dos o más desarrolladores realizan cambios simultáneos en el mismo archivo y esos cambios son incompatibles entre sí. Es decir, los cambios realizados por diferentes personas afectan partes del archivo que no pueden ser fusionadas automáticamente.

Tipos de Conflictos:

1. **Cambio simultáneo en la misma línea**: Dos desarrolladores modifican la misma línea de código.
2. **Cambio en distintas líneas del mismo archivo**: Cambian diferentes líneas, pero las modificaciones afectan la estructura del archivo.
3. **Ramas no fusionadas correctamente**: Si se trabaja en ramas separadas, pero no se logra combinar correctamente el trabajo.

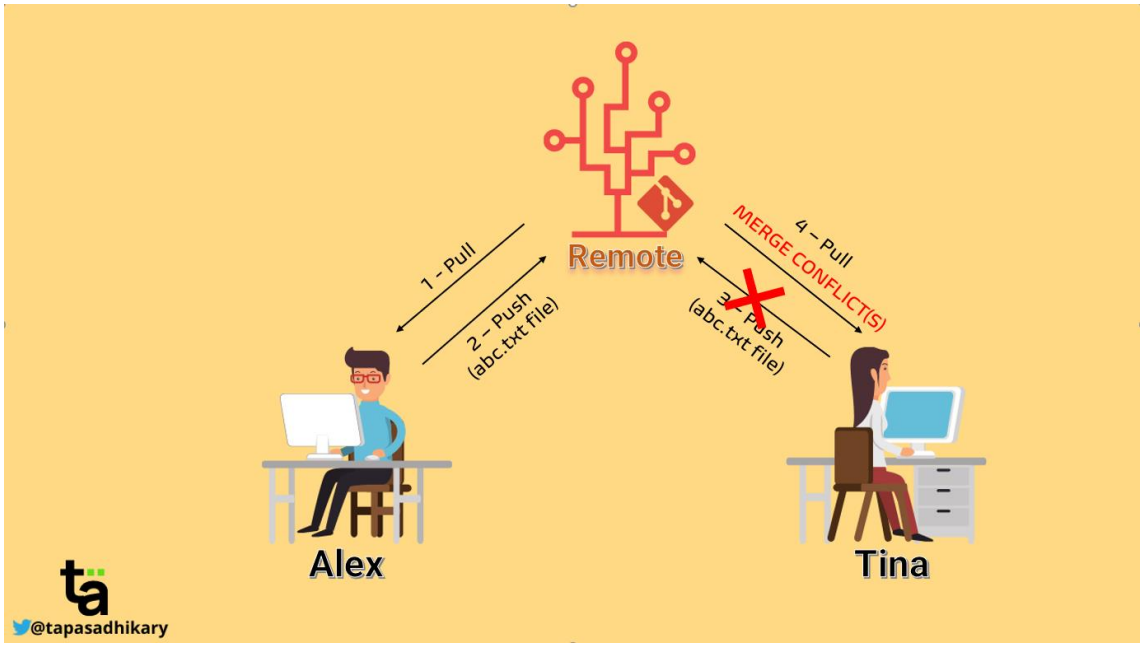
Ejemplo de Conflicto:

- **Usuario A** agrega una función.
- **Usuario B** edita otra función en el mismo archivo.
- Ambos hacen commits y quieren fusionar sus ramas principales.

Si el sistema no detecta estos cambios, puede producir un **conflicto** porque los cambios de ambos no son compatibles.

Resolución de Conflictos:

Cuando se detecta un conflicto, el sistema de control de versiones no permite continuar con la fusión automática de ramas. En lugar de eso, muestra un mensaje indicando dónde ocurrió el conflicto.



Paso a paso para resolver conflictos:

1. **Identificación**: El SCV muestra los archivos con conflictos.
2. **Resolución manual**: Editas manualmente el archivo para combinar los cambios.
3. **Commit manual**: Confirmas los cambios una vez que se ha solucionado el conflicto.
4. **Revisar**: Asegúrate de que los cambios fusionados no afecten la lógica del proyecto.
5.

Ejemplo Práctico con **Git**:

1. Dos desarrolladores trabajan en un archivo (`archivo.txt`).
2. Usuario A hace un cambio: agrega una nueva línea.
3. Usuario B cambia otra línea.
4. Cuando fusionan sus ramas, Git detecta el conflicto porque ambas ediciones afectan el mismo archivo.
5. Git muestra las diferencias y sugiere cómo resolverlo.
6. Los desarrolladores resuelven el conflicto, editan el archivo manualmente, y hacen commit.

Herramientas de Resolución de Conflictos:

- **Git** usa herramientas visuales como `git mergetool`.
- **SVN** ofrece herramientas de línea de comando para fusionar manualmente.
- **Perforce** permite ediciones simultáneas en su interfaz.

Introducción a GitHub: ventajas, características, funciones de revisión.

Introducción a GitHub

GitHub es una plataforma de desarrollo colaborativo basada en Git, un sistema de control de versiones distribuido. Es ampliamente utilizado para alojar proyectos de software, facilitar la colaboración entre desarrolladores y gestionar el ciclo de vida del desarrollo de software.

git	GitHub
Herramienta de control de versiones distribuida	Plataforma basada en la nube
Herramienta de código abierto que los desarrolladores instalan localmente para gestionar el código fuente	Servicio en línea al que los desarrolladores que utilizan Git pueden conectarse y cargar o descargar recursos

Ventajas de GitHub

1. **Colaboración eficiente:**
   Permite que múltiples desarrolladores trabajen en el mismo proyecto de manera simultánea mediante la gestión de ramas, solicitudes de extracción (pull requests) y revisiones de código.
2. **Alojamiento centralizado de repositorios:**
   Proporciona un lugar seguro y accesible para almacenar código fuente, documentos y recursos relacionados con un proyecto.
3. **Control de versiones:**
   Ofrece herramientas avanzadas para gestionar cambios en el código, facilitando el rastreo del historial y la reversión a versiones anteriores si es necesario.
4. **Integración con herramientas de CI/CD:**
   GitHub se integra con servicios como GitHub Actions para implementar pipelines automatizados de integración y entrega continua.
5. **Comunidad activa:**
   Millones de desarrolladores comparten proyectos de código abierto, lo que facilita el aprendizaje y el uso de herramientas ya existentes.
6. **Transparencia y colaboración global:**
   Puedes acceder a proyectos de otros desarrolladores, contribuir a ellos y obtener retroalimentación.

Características principales de GitHub

1. **Repositorios:**
   Son los contenedores donde se almacena el código y los archivos relacionados con un proyecto.
2. **Ramas (branches):**
   Permiten trabajar en diferentes características o correcciones de manera aislada del código principal.
3. **Pull Requests (PR):**
   Facilitan la colaboración al permitir que los cambios propuestos sean revisados antes de fusionarse con la rama principal.
4. **Issues:**
   Herramientas para rastrear errores, sugerencias y tareas pendientes dentro del proyecto.
5. **Wiki:**
   Espacio donde los equipos pueden documentar el proyecto, guías, y otros recursos útiles.
6. **GitHub Pages:**
   Permite alojar sitios web directamente desde un repositorio de GitHub.
7. **Acciones de GitHub (GitHub Actions):**
   Sistema de automatización para crear flujos de trabajo, como pruebas automáticas o despliegues.
8. **Integración con herramientas externas:**
   Compatibilidad con IDEs, servicios de análisis de código y herramientas de gestión de proyectos.

Funciones de revisión en GitHub

1. **Comentarios en Pull Requests:**
   Permite dejar observaciones sobre líneas específicas de código, mejorando la comunicación entre desarrolladores.
2. **Aprobación o rechazo de cambios:**
   Los revisores pueden aprobar los cambios propuestos, solicitando modificaciones si es necesario.
3. **Protección de ramas:**
   Requiere que las ramas principales (como la rama `main`) incluyan revisiones antes de aceptar cambios, asegurando la calidad del código.
4. **Comparación de cambios (diffs):**
   Muestra las diferencias entre las versiones del código para que los revisores puedan analizar qué se ha modificado.
5. **Solicitudes de cambios:**
   Los revisores pueden pedir ajustes específicos antes de fusionar la rama, ayudando a mantener altos estándares de calidad.
6. **Integración con herramientas de calidad:**
   Puede usar verificaciones automáticas como análisis de estilo, pruebas unitarias y pruebas de integración antes de aceptar.